

Práctica 1 Diseño sostenible y principios SOLID

Fase 1 - Diagnóstico SOLID del código legado

Contexto: El módulo legado concentra toda la lógica en la God Class

GestorCitasOdontologicas (51 líneas lógicas, 7 responsabilidades, complejidad ciclomática 10 en AgendarCita) y la acopla directamente a SQL Server, SMTP y Twilio, además de tener credenciales escritas en el código. Se diagnosticaron violaciones de los cinco principios SOLID, se midió la línea base y se refactorizó hacia una arquitectura por capas con inyección de dependencias.

1. Matriz de Violaciones SOLID:

Principio	Dónde	Cómo se viola	Consecuencia
S — Responsabilidad única	GestorCitasOdontologica.s.cs L5–98 (toda la clase) NotificacionServicio.cs L12–22 Cita.cs L16–18	La clase tiene 7 razones de cambio: (1) validar disponibilidad L14–17; (2) calcular copago L19–57; (3) construir la cita e ID L58–66; (4) calcular multa L77–83; (5) persistir L68 y L88; (6) notificar L69 y L89; (7) acumular estadísticas L9–10, L71, L91, L95–97. Además muta el estado de la entidad (L85–86). NotificacionServicio mezcla correo y SMS en un método; la entidad Cita redacta el texto de la notificación.	Un cambio de tarifa, de proveedor SMTP o de esquema SQL obliga a editar y volver a probar la misma clase, lo cual tiene alto riesgo de regresión
O — Abierto/cerrado	GestorCitasOdontologica.s.cs L22–37, L40–47, L49–57, L79–83 Odontologo.cs L11; Paciente.cs L13	Cadenas if / else if sobre números mágicos: 4 especialidades (L22–37), 3 convenios (L40–47), recargos (L49–57) y multa especial de cirugía (L79–83). Los códigos 1, 2, 3, 4 solo están "documentados" con comentarios. Los factores (1.2m, 1.8m, 2.5m, 1.1m, 0.30m, 0.10m, 20, 35, 50, 40) están incrustados	Cada nueva especialidad, convenio o regla exige modificar el código ya probado. Una especialidad desconocida produce copago 0 sin avisar (L20 nunca se actualiza).
L — Sustitución de Liskov	No hay herencia en el proyecto. Violaciones de contrato: NotificacionServicio.cs L12 vs L17 GestorCitasOdontologica.s.cs L12 vs L81, L16	No existe una violación formal por subtipado (no hay jerarquías). Sí hay violaciones del contrato de comportamiento: (a) EnviarEmailYSms(cita, mensaje) ignora mensaje y siempre envía "Su cita quedó programada...": la cancelación (L89 del Gestor) notifica que la cita quedó programada; (b) la especialidad llega por dos vías que pueden contradecirse: parámetro tipoEspecialidad (L12) y odontologo.EspecialidadId (L81); (c)	Ningún colaborador puede sustituirse con garantías (ni siquiera un doble de prueba). El riesgo es latente: apenas se introduzca una abstracción o herencia sobre estas clases, el contrato roto aparece

		se lanza Exception genérica (L16), imposible de distinguir o sustituir.	
I — Segregación de interfaces	GestorCitasOdontologica.s.cs L12, L75, L95, L97 NotificacionServicio.cs L12 Program.cs L48	No existen interfaces: la superficie pública del Gestor es una "interfaz gorda" implícita que mezcla comandos (agendar, cancelar) y reportes (totales). Un cliente que solo quiere los totales (Program.cs L48) depende de toda la clase y, transitivamente, de SQL Server y SMTP. EnviarEmailYSms obliga a usar ambos canales aunque se necesite uno.	Los clientes quedan acoplados a métodos que no usan; no se puede reutilizar solo el correo, solo el SMS ni solo la consulta.
D — Inversión de dependencias	GestorCitasOdontologica.s.cs L1, L7–8, L68–69, L88–89 SqlServerEjecutor.cs L10 NotificacionServicio.cs L10–11, L14–15, L21	El módulo de alto nivel crea clases concretas de bajo nivel con new (L7: SqlServerEjecutor, L8: NotificacionServicio). Las credenciales están en código: contraseña de sa (SqlServerEjecutor L10), servidor SMTP y API key de Twilio (NotificacionServicio L10–11); la API key se imprime en consola (L21). SmtClient y SqlConnection se instancian dentro de los métodos.	Es imposible probar el Gestor sin SQL Server y SMTP reales; cambiar de proveedor es editar la lógica de negocio y hay riesgo de seguridad por secretos expuestos.

2. Línea Base Cuantitativa (Baseline):

a. LCOM96b:

$$LCOM96b = \frac{1}{a} * \sum_{j=1}^a \left[\frac{m - \mu(A_j)}{m} \right]$$

Paso 1. Atributos (a = 4):

A₁ = _db

A₂ = _notificador

A₃ = _totalRecaudadoMes

A₄ = _totalCitasCanceladas

Paso 2. Métodos (m = 4):

M₁ = AgendarCita

M₂ = CancelarCita

M₃ = ObtenerTotalRecaudado

M₄ = ObtenerTotalCanceladas

Paso 3. Qué métodos usan cada atributo:

A₁ _db: lo usan AgendarCita (L68) y CancelarCita (L88). $\mu(A_1) = 2$

A₂ _notificador: lo usan AgendarCita (L69) y CancelarCita (L89). $\mu(A_2) = 2$

A₃ _totalRecaudadoMes: lo usan AgendarCita (L71) y ObtenerTotalRecaudado (L95). $\mu(A_3) = 2$

A₄ _totalCitasCanceladas: lo usan CancelarCita (L91) y ObtenerTotalCanceladas (L97). $\mu(A_4) = 2$

Paso 4. Sustitución:

$$LCOM96b = (1/4) \times [(4-2)/4 + (4-2)/4 + (4-2)/4 + (4-2)/4]$$

$$LCOM96b = (1/4) \times [0.50 + 0.50 + 0.50 + 0.50]$$

$$LCOM96b = (1/4) \times 2.00$$

$$LCOM96b = 0.50$$

Interpretación: el valor va de 0 (máxima cohesión, todos los métodos usan todos los atributos) a 1 (ninguna cohesión). Con 0.50, cada atributo lo usa solo la mitad de los métodos. Las dependencias de infraestructura (_db y _notificador) las comparten solo Agendar y Cancelar, y cada contador pertenece a un único flujo.

b. Acoplamiento Ca y Ce:

$$Ca = 1$$

-Dirección: Entrante

- Program: crea y usa el Gestor (new GestorCitasOdontologicas()). Es la única clase del proyecto que lo referencia.

$$Ce = 5$$

-Dirección: Saliente

1. SqlServerEjecutor (L7, L68, L88): clase concreta de infraestructura.
2. NotificacionServicio (L8, L69, L89): clase concreta de infraestructura.
3. Cita (L12, L58, L75): clase del modelo.
4. Paciente (L12): clase del modelo.
5. Odontologo (L12): clase del modelo.

Nota: solo se cuentan clases del proyecto. Los tipos del framework (Exception, DateTime, TimeSpan, Guid) no se cuentan.

c. Inestabilidad(I), Abstracción(A) y Distancia(D)

$$I = \frac{C_e}{C_e + C_a},$$

$$A = \frac{\sum m_a}{\sum m_c}, \quad D = |A + I| - 1$$

Donde:

Ca y Ce son los valores de 2.b (1 y 5 respectivamente)

m_a = métodos abstractos (declarados sin implementación, en interfaces o clases abstractas)

m_c = total de métodos de la clase

Inestabilidad: $I = 5 / (5 + 1) = 5 / 6 = 0.8333$

Abstracción: $A = 0 / 4 = 0.00$ (la clase es concreta: sus 4 métodos tienen implementación y no hay ninguno abstracto)

Distancia: $D = |0.00 + 0.8333 - 1| = |-0.1667| = 0.1667$

Nota: el enunciado escribe $D = |A + I| - 1$, pero con esa expresión el resultado sale negativo (-0.1667). Se usó la definición estándar, $D = |A + I - 1|$.

Posición en el plano A vs. I: (I = 0.83, A = 0.00).

¿Cae en la Zona de Dolor (A = 0, I = 0)? La clase por sí sola no: tiene I = 0.83, es decir, es concreta e inestable, porque depende de mucho y casi nada depende de ella.

Esa posición queda cerca de la secuencia principal (D = 0.17). Si se analiza el módulo completo DentaCare.Legacy como un solo componente, sí cae en la Zona de Dolor: Ca = 1 (Program), Ce = 0 (todas sus dependencias son internas), I = 0, A = 0 (ningún método abstracto) y D = 1.00. Es concreto, estable y sin ninguna abstracción por donde extenderlo.

Fase 3: Recálculo de Métricas e Impacto Cuantitativo

Aspecto	Métrica	Antes	Después	Virtud lograda
Cohesión (SRP)	LCOM96b de la clase principal	0.50	0.00	Cada clase tiene una sola razón de cambio
Cohesión (SRP)	LCOM96b promedio de las clases con estado	0.167 (3 clases)	0.028 (12 clases)	La cohesión mejora en todo el módulo
Cohesión (SRP)	Responsabilidades de la clase principal	7	1	La God Class se divide en clases pequeñas
Acoplamiento	Ca	1	1	Sin cambio: solo la usa la raíz de composición
Acoplamiento	Ce	5	11	Sube porque las dependencias ahora son explícitas; todas son interfaces o modelo
Acoplamiento (DIP)	Ce hacia clases concretas de infraestructura	2	0	Se depende de abstracciones, no de SQL ni SMTP
Estabilidad	$I = Ce / (Ce + Ca)$	0.83	0.92	Ambas son clases concretas e inestables, lo esperable
Abstracción	A de la clase	0.00	0.00	La abstracción ahora vive en las interfaces que recibe

Distancia	D de la clase	0.17	0.08	Más cerca de la secuencia principal
Distancia del módulo	D del módulo completo	1.00 (Zona de Dolor)	0.22 (ponderada por componentes)	Salir de la Zona de Dolor
Abierto/cerrado (OCP)	Costo de agregar una especialidad	Editar el Gestor	1 clase nueva + 1 línea de registro	Se extiende sin modificar clases de negocio
Segregación (ISP)	Interfaces del módulo	0	16	Cada cliente depende solo de lo que usa
Sustitución (LSP)	Implementaciones intercambiables	Ninguna	SQL ↔ memoria, SMTP/Twilio ↔ consola	Los contratos se respetan y se pueden sustituir
Testabilidad (DIP)	Probar sin SQL ni SMTP	No es posible	Es posible	Los servicios aceptan dobles de prueba